

SYSTEM DESIGN DOCUMENT · DRAFT V0.1

PARALLAX.

A cross-venue, multi-asset, event-driven statistical arbitrageur for prediction markets — built on borrowed strategy principles, never borrowed trades, with its edge concentrated in signal speed, normalization correctness, and execution quality rather than infrastructure that these venues don't offer.

Polymarket

Kalshi

+N future venues

Weather · Econ · News · Oracle alpha

Status

Draft for engineering review

Date

2026-08-12

Scope

Architecture, latency budget, testing framework

CONTENTS

FRAMING

- Executive summary
- Reality check
- Strategy DNA

CORE PIPELINE

- Architecture
- Data ingestion
- Consolidated book
- Alpha models
- Strategy core

EXECUTION INFRA

- Execution & adapters
- Risk engine
- Self-calibration

Technology stack

Latency budget

Deployment

VALIDATION

Testing framework

Compliance guardrails

Roadmap

Closing note

01

Executive summary

PARALLAX is a cross-venue, multi-asset, event-driven statistical arbitrageur for prediction markets. It holds a single unifying capability that no single referenced project has on its own: it fuses domain-specific probabilistic forecasting (the PolyWeather / Kalshi-Weather-Model lineage), grid-based liquidity provision (`polymarket_lp_tool`), active two-sided market making with inventory control (`CloddsBot`), and a modular plugin architecture (the `HarrierOnChain` toolkits) into one strategy core that treats every venue's order book as one noisy observation of a single underlying probability.

Its edge is not "faster than everyone." It is **faster to a correct answer**: it normalizes semantically identical contracts across venues into one canonical representation, prices them with domain models the moment new information becomes machine-readable, and routes capital — as a maker, a taker, or a cross-venue arbitrageur — to whichever venue is currently wrong relative to the other two.

The name is deliberate: like optical parallax, the strategy's core signal is the apparent disagreement between two vantage points on the same object. A weather station doesn't move; two markets pricing the same weather event should not disagree by more than the cost of being wrong. When they do, that gap is the trade.

3

STRATEGY ENGINES

4

ALPHA DOMAINS

<10 μ s

INTERNAL DECISION PATH

2

LIVE VENUES AT LAUNCH

02

Reality check on "ultra-low latency"

Before specifying anything else, the infrastructure premise needs correcting, because it determines where every subsequent engineering hour goes. The brief calls for colocation, kernel-bypass networking, FPGA-accelerated parsing, FIX over dedicated cross-connects, and a sub-10-microsecond *end-to-end* critical path. None of that exists for Polymarket or Kalshi today, and building as though it does would waste the budget on infrastructure with nothing to plug into.

ENGINEERING REALITY CHECK

Polymarket exposes a REST + WebSocket CLOB API. Matching happens off-chain on Polymarket's own operator infrastructure; settlement finalizes on Polygon in batches. There is no colocation program, no FIX gateway, and no binary multicast feed to accelerate with an FPGA — the wire format is JSON over WebSocket.

Kalshi, despite being a CFTC-regulated exchange, offers REST + WebSocket market data and order entry to API users. It does not publish a FIX gateway or colocation service for retail/algorithmic access as of this writing.

FPGA parsing and kernel-bypass NICs earn their keep when the bottleneck is *local* — decoding a binary multicast feed off a colocated switch faster than a neighboring rack. Here the bottleneck is the public internet round trip and each venue's own application-layer processing, which no amount of local silicon shortens. Spending the engineering budget there would be optimizing a part of the pipeline that isn't the constraint.

The honest reframe, which this document specifies in full: split the latency budget into two regimes that behave completely differently, and be excellent at each on its own terms.

- **Internal decision path** — from a normalized tick landing in memory to a fully-formed order object ready to serialize onto the wire. This is entirely under our control, has no network variance, and **<10 microseconds is a legitimate, achievable target** with a lock-free Rust hot path (see §13).
- **External wire path** — network transport plus each venue's own matching/ack latency. This is bounded below by physics and by infrastructure PARALLAX does not control. The achievable lever here is *proximity and connection hygiene*: run compute in the cloud region closest to each venue's API ingress, hold warm persistent connections, resume TLS sessions, and never pay a fresh handshake on the trade path. Realistic figures are single-to-double-digit milliseconds, not microseconds.

Everywhere the original brief says "microseconds" about something that touches the network or a venue's servers, read it as "the fastest achievable given public, rate-limited, JSON-over-HTTP(S)/WebSocket APIs, revisited the moment either venue offers institutional connectivity." Everywhere it says "microseconds" about PARALLAX's own code, that target stands and is specified precisely.

03

Strategy DNA — principles, not trades

Each reference project contributes a *principle*, generalized and re-implemented from scratch. PARALLAX never observes or conditions on another trader's orders, wallet activity, or fills. The only permissible inputs to an order decision are PARALLAX's own alpha estimates, the consolidated book state, and its own risk/inventory state — enforced at the type level, not just by policy (see §8).

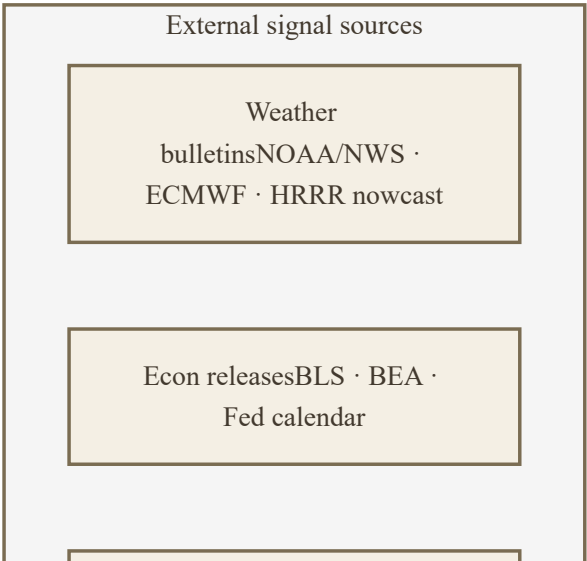
SOURCE	PRINCIPLE OBSERVED	HOW PARALLAX GENERALIZES IT
CloddsBot	Two-sided market making around a computed fair price.	Core MM engine quotes off a <i>consolidated cross-venue</i> fair price rather than a single venue's last trade: quote

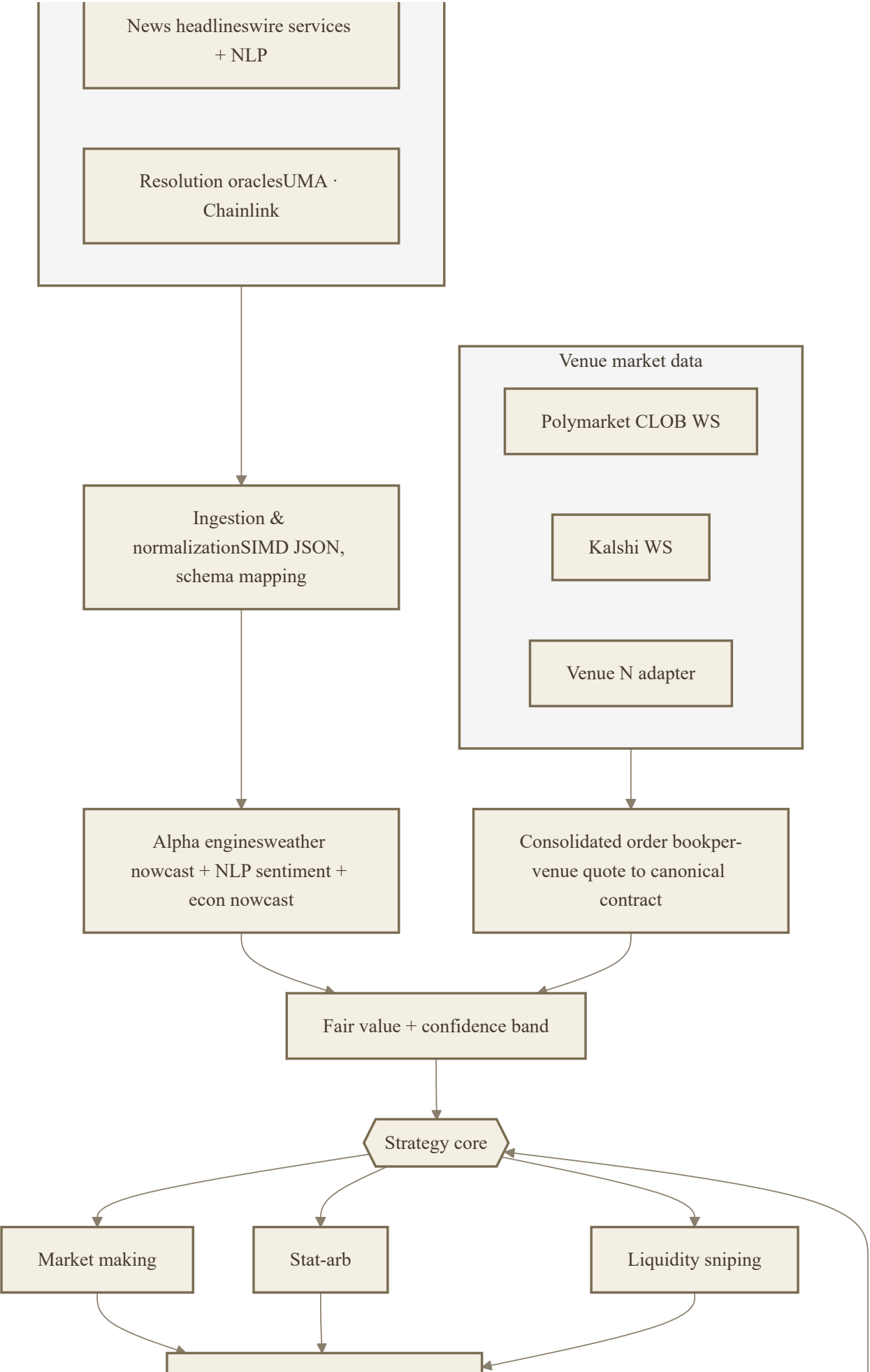
	with active inventory management	width and skew are driven by live inventory and the alpha model's confidence band, not a fixed spread
PolyWeather / Kalshi-Weather-Model	Domain-specific probabilistic forecasting used as an alpha source independent of order flow	Generalized into a pluggable AlphaSource interface; weather is one implementation, econ-release nowcasting and NLP headline sentiment are siblings on the same interface
polymarket_lp_tool	Grid-based liquidity provision — laddered resting orders capturing spread across a price grid	Folded into the MM engine's ladder generator, extended to dynamically choose <i>which venue's grid</i> has the better economics at any given moment rather than being fixed to one
HarrierOnChain toolkits	Modular, composable toolkit architecture for on-chain data and execution	Generalized into PARALLAX's plugin boundary — AlphaSource , VenueAdapter , and ExecutionAlgo traits — so new signal sources, venues, and execution styles plug in without touching the hot path (see §17)

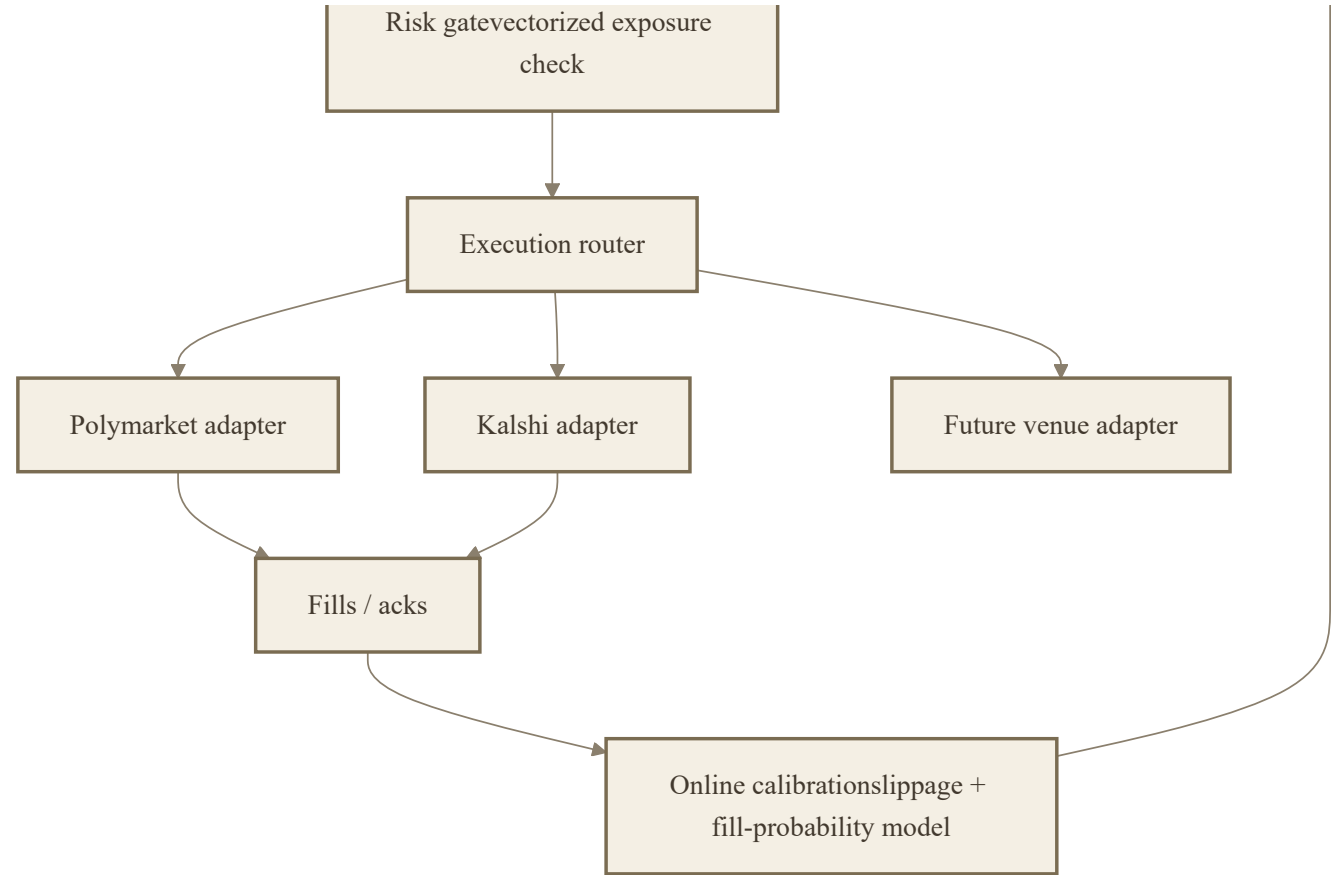
04

System architecture

PARALLAX is event-sourced end to end: every external message, internal signal, risk decision, and order/fill is an immutable, timestamped event appended to a log before anything acts on it. State — positions, quotes, model parameters — is a fold over that log, which is what makes hot-hot redundancy (§14) and deterministic replay testing (§15) tractable rather than aspirational.







Two properties matter most here. First, **alpha and book are separate tributaries** that only merge at the fair-value stage — a weather model never talks to an order book directly, which keeps each AlphaSource testable in isolation. Second, **every strategy engine terminates at one risk gate** before anything reaches a venue, so market making, stat-arb, and sniping can never collectively exceed the position limits even though each is written as if it owned the book.

05

Data ingestion layer

Each external source has a genuinely different latency profile, and the ingestion layer is built per-source rather than through one generic poller, because treating a scheduled government release the same as a continuous WebSocket feed wastes engineering effort in both directions.

SOURCE	CADENCE	NATIVE LATENCY FLOOR	INGESTION APPROACH
Weather bulletins	Continuous / scheduled	Minutes (NWP model cycles); seconds for point observations	Poll NDFD/HRRR endpoints on model-run schedule; hold a standing ensemble in

model runs		memory, recompute nowcast deltas on each new observation, not on a timer	
Econ releases	Scheduled to the second (e.g., 08:30 ET)	Sub-second at the publisher; consumer-side is a race	Pre-warm connections before the scheduled release; parse the release payload with a hand-built schema per series (no generic JSON walk) to shave parse time when it matters most
News headlines	Continuous, bursty	Seconds from wire to consumer	Persistent stream connection; lightweight on-path NLP (entity + polarity) sized to run in low-microsecond territory per headline, heavier sentiment models run off-path and feed back asynchronously
Resolution oracles	Event-driven, irregular	Block time / oracle challenge windows (can be minutes)	Treated as a slow, high-confidence signal — not latency-critical, but decisive: an oracle update can collapse a market's confidence band to near zero
Venue order books	Continuous, push-based WS	Venue-dependent, tens of ms	One persistent WS connection per venue per instance, SIMD JSON parse into a pre-allocated normalized tick struct, zero heap allocation on the parse path

The unifying discipline: every source, regardless of cadence, emits the same internal `RawEvent` envelope (source id, publish timestamp, receive timestamp, payload) onto the same ring buffer, so the strategy core never special-cases where a fact came from — only how confident it is and how stale it's allowed to get before the confidence band widens.

06

Consolidated cross-venue order book

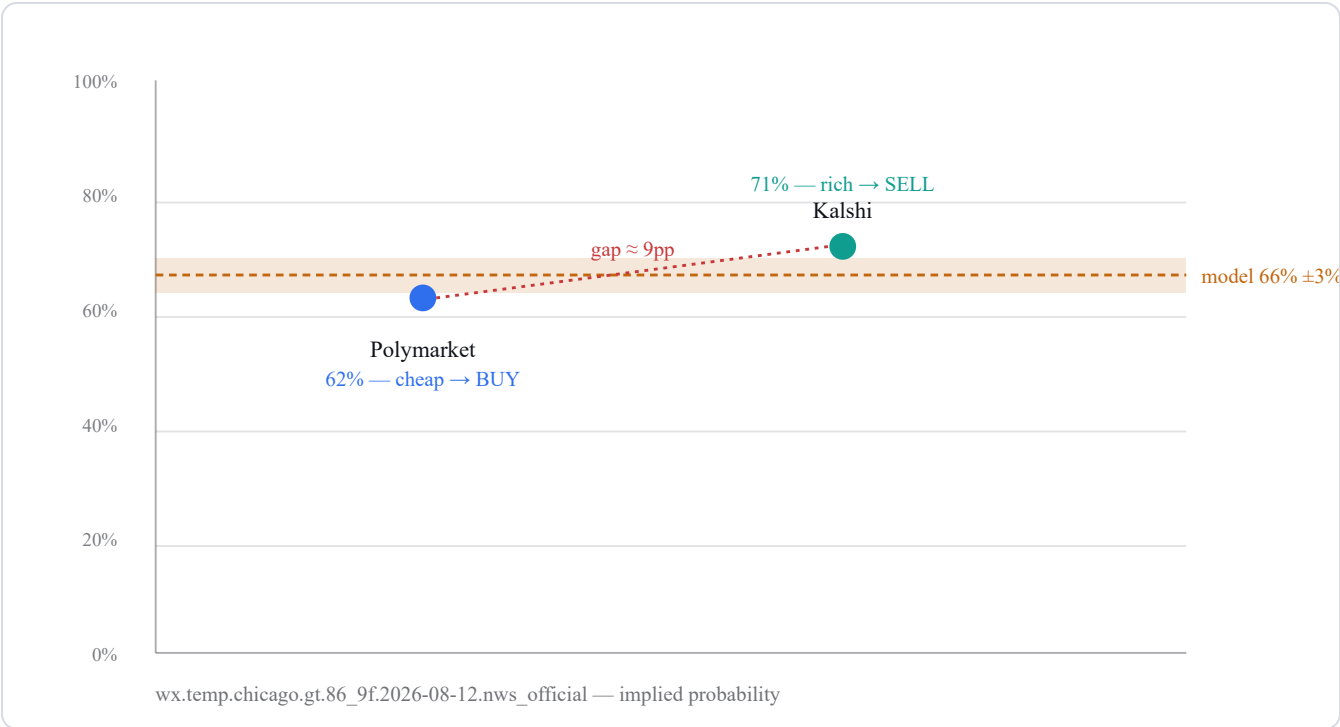
The hardest correctness problem in this system isn't speed — it's knowing that two listings on two venues are actually the same bet. Venues differ in units (°C vs °F), strike framing (single threshold vs. bucketed ranges), expiry granularity, and settlement source. PARALLAX maps every venue listing to a canonical contract id:

(event class, location, threshold, direction, resolution window, resolution source) . with per-

venue adapters responsible for the unit and framing conversion at ingestion time, not downstream.

VENUE LISTING	NATIVE FRAMING	CANONICAL MAPPING
Polymarket — "Chicago temperature > 30°C, Aug 12"	Metric threshold, midnight UTC cutoff	wx.temp.chicago.gt.86_9f.2026-08-12.nws_official
Kalshi — "Chicago high > 86°F, Aug 12"	Imperial threshold, local-day NWS climatological cutoff	

Once two listings resolve to the same canonical id, PARALLAX can legitimately compare their implied probabilities and treat any persistent gap as a relative-value opportunity — this is the mechanism the whole stat-arb engine rests on.



Same canonical contract, two venues, one gap: Polymarket trades cheap and Kalshi trades rich relative to PARALLAX's fair-value band. The stat-arb engine only acts once a venue price sits fully outside the band — not merely away from the midpoint.

fair value is an ensemble, not a single model. Each AlphaSource emits a probability estimate plus its own uncertainty; the aggregator combines them with inverse-variance weighting so a confident weather nowcast dominates a noisy sentiment read, and vice versa when the situation reverses (e.g., a market resolving on an economic release, where NLP sentiment on the release headline briefly dominates a now-irrelevant weather signal).

ALPHA DOMAIN	MODEL SKETCH	OUTPUT
Weather nowcast	Ensemble spread across NWP model runs (HRRR, GFS, ECMWF) reweighted toward the most recent cycle; disagreement between ensemble members <i>is</i> the uncertainty term, not a separate estimate	P(threshold met) + std. dev. from ensemble spread
Econ nowcast	Consensus-vs-actual delta scored against historical surprise distribution for that series	P(threshold met) + surprise-conditioned confidence
NLP sentiment	Lightweight on-path classifier (polarity + entity match) for immediate reaction; heavier off-path model refines within seconds and revises the estimate	Directional nudge + decaying confidence (headline relevance fades fast)
Oracle state	Direct read of resolution-source state once available	Near-certainty override — collapses the confidence band when triggered

The aggregate feeds a single number the rest of the system consumes: **fair value ± dynamic confidence band**. The band widens with model disagreement, data staleness, and low sample-size ensemble spread — this is what stops the stat-arb engine from firing on noise and is precisely the "weather-forecast disagreement as signal" principle generalized beyond weather.

08

Strategy core

Three engines run continuously against the same fair-value stream and consolidated book, each with a distinct trigger condition, and all three route through the single

risk gate before anything reaches a venue.

Market making

Posts two-sided, ladder quotes (the polymarket_lp_tool grid principle) centered on the consolidated fair value rather than either venue's own mid. Quote width is a function of the confidence band (wider band → wider quotes) and current inventory (skewed away from the side that would increase exposure). The engine chooses which venue's grid to stand in — sometimes both — based on the calibration layer's live fill-probability and fee estimates per venue (§11).

Stat-arb

Fires only when a venue's price sits fully outside the fair-value confidence band — not merely off the midpoint. Position size follows a fractional-Kelly cap bounded by the risk gate's per-event and per-correlated-cluster limits (many weather contracts on the same city/date are highly correlated and must be sized as one exposure, not summed independently).

Liquidity sniping

Triggers only as a direct, immediate consequence of PARALLAX's own alpha recomputation — a resting quote on either venue that the fresh fair value now judges mispriced. This is the one place latency inside the internal decision path (§13) has a direct, measurable payoff: the faster the recompute-to-order path, the more of that stale-quote window is captured before it updates or someone else takes it.

ARCHITECTURAL ENFORCEMENT OF "NO FOOTPRINT FOLLOWING"

The strategy core's input type is `StrategyInput { fair_value: FairValue, book: ConsolidatedBook, inventory: Position }`. There is no field for observed counterparty orders, wallet activity, or flow data. If a future `AlphaSource` ever ingests on-chain flow as a research signal, its output is still just a `ProbabilityEstimate` like any other source — it cannot special-case "do what that wallet did," because the type the strategy core consumes has no slot for it.

Execution & venue adapters

Each venue is a plugin behind one trait, so adding venue N+1 means implementing an adapter, not touching the strategy core.

```
pub trait VenueAdapter: Send + Sync {
    fn venue_id(&self) -> VenueId;
    fn subscribe(&self, contracts: &[CanonicalContractId]) -> BookStream;
    fn submit(&self, order: OrderIntent) -> Result<OrderAck, ExecError>;
    fn cancel(&self, order_id: OrderId) -> Result<(), ExecError>;
    fn capabilities(&self) -> VenueCapabilities; // rate limits, tick size, fee schedule
}
```

VENUE	ORDER ENTRY	KNOWN CONSTRAINTS TO DESIGN AROUND
Polymarket	EIP-712 signed orders submitted to the off-chain CLOB operator	Signing must be pre-computed where possible (template orders, sign on size/price fill-in only); on-chain settlement batches mean matching confirmation and final settlement finality are two different latencies — only the former is on PARALLAX's decision-to-ack path
Kalshi	REST order entry + WS trade/fill confirmation	Published rate limits must be respected with headroom, not skated against; REST round trip dominates the wire budget
Future venues	Adapter TBD	Bring-up checklist: rate limits, tick size, fee schedule, settlement model, geofencing — captured once in <code>VenueCapabilities</code> and consumed by risk, calibration, and the smart order router uniformly

The **smart order router** picks the venue (or splits across both) using live fill-probability and slippage estimates from the calibration layer, not a static preference — the same relative-value gap can sometimes be captured better by resting on the cheap side than by crossing the spread on the rich side.

10

Risk engine

Every proposed order from every engine passes one vectorized risk gate before it

reaches an adapter. "vectorized" here means the exposure recompute is a batch operation over the position vector (SIMD-friendly fixed-point arithmetic, no per-instrument branching), not that it is symbolic or approximate — correctness on the risk path is non-negotiable even when everything upstream is a probabilistic estimate.

- **Correlated-cluster netting** — contracts sharing an underlying event (e.g., three temperature thresholds on the same city/date) are risk-netted as one exposure, not summed as independent bets.
- **Per-venue and per-event limits** — hard caps that the strategy core cannot request past, checked before order construction completes.
- **Kill switches** — feed staleness beyond a threshold, venue error-rate spikes, or realized-vs-expected slippage drift beyond calibration bounds each independently flatten the affected instrument set and halt new quoting on it.
- **Latency-aware throttling** — if the internal decision path (§13) drifts above its budget (GC-free Rust shouldn't drift, but queueing under burst load can), quoting aggressiveness backs off automatically rather than quoting blind.

11

Self-calibration

A lightweight online model — recursive least squares or a small contextual bandit, deliberately *not* a heavy retraining loop — tracks realized slippage and fill probability per venue, per contract class, per order size, and per time-of-day. It updates incrementally on every fill or reject, entirely off the hot path (asynchronously, batched), and feeds two things back into the strategy core: quoting aggressiveness (how far to lean into the spread) and venue choice weighting in the smart order router.

Heavier model fitting — the weather ensemble's own weighting scheme, the sentiment classifier's parameters — is offline work in Python, versioned and shipped as a config/weights artifact the hot path loads, never trained in production.

12

Technology stack

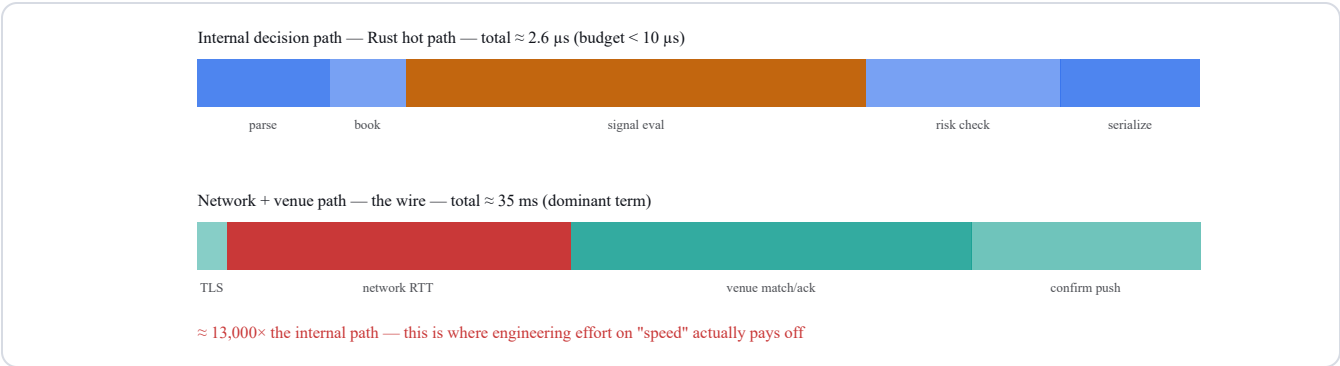
LAYER	TECHNOLOGY	RATIONALE
Hot path core	Rust, thread-per-core, simd-json / rkyv , lock-free SPSC/MPSC ring buffers (Disruptor pattern)	Predictable low-microsecond latency, no GC pauses, memory safety without a runtime tax
Internal event bus	Shared-memory ring (e.g. iceoryx2) for same-host fan-out; NATS JetStream for cross-process, non-critical consumers	Keeps monitoring/backtest/logging consumers off the trade-critical path entirely
Venue connectivity	Rust async WS/REST clients, persistent connections, TLS session resumption, HTTP/2 keep-alive	The only real lever against wire latency given each venue's public API surface
Offline modelling	Python — polars/pandas, PyTorch/statsmodels, scikit-learn, Jupyter	Fast iteration for weather ensembles and NLP models; not latency-sensitive, optimized for research velocity
Backtest/replay engine	Python orchestration around the same compiled Rust strategy core	Guarantees backtest/live code-path parity — see §15
Tick/event storage	Columnar time-series store (QuestDB / ClickHouse)	High-ingest, fast range scans for replay and post-trade analysis
Reference/state storage	PostgreSQL + append-only event log to object storage	Durable, auditable state reconstruction via replay
Observability	Prometheus/Grafana, async structured tracing with nanosecond timestamps	Exported off-path only; must never block the trade loop
Deployment	Bare-metal or Kubernetes, hot-hot across two cloud regions/AZs	Redundancy without a single point of failure (§14)

13

Latency budget

Two regimes, honestly separated, as promised in §2. The internal path is where the

sub-10-microsecond target is real and specified stage-by-stage; the wire path is where physics and each venue's own processing dominate.



Same visual width, wildly different scales: the internal path (top) is genuinely microsecond-class and fully within PARALLAX's control; the wire path (bottom) is millisecond-class and bounded mostly by each venue's own infrastructure. Total realistic tick-to-trade is wire-dominated at roughly 10–60 ms depending on venue and path.

STAGE	REGIME	TYPICAL	BUDGET CEILING
Deserialize normalized tick	Internal	~350 ns	1 μs
Consolidated book update	Internal	~200 ns	0.5 μs
Signal evaluation (vectorized)	Internal	~1.2 μs	4 μs
Risk gate check (vectorized)	Internal	~0.5 μs	2 μs
Order object construction	Internal	~0.35 μs	1.5 μs
Internal subtotal		~2.6 μs	< 10 μs
TLS session reuse overhead	Wire	~1 ms	2 ms
Network round trip (region-matched)	Wire	5–15 ms	25 ms
Venue matching / internal ack	Wire	5–20 ms	40 ms
Confirmation propagation	Wire	5–10 ms	20 ms
Wire subtotal		~16–46 ms	< 90 ms

Techniques that move the internal number: core pinning and NUMA-aware

allocation, zero-copy/zero-allocation parsing, lock-free ring buffers, no syscalls on the trade path. Techniques that move the wire number: region selection by measured RTT (not assumption), persistent connections, TLS resumption, and pre-templated order payloads that only need size/price fields filled at decision time.

14

Deployment & redundancy

Hot-hot across two regions/AZs, both instances continuously computing signals against the same event-sourced log. A per-venue leader lease (etcd/Raft-style) determines which instance actually submits orders to a given venue at a given moment, chosen by whichever instance is currently observing the lower measured RTT to that venue's ingress — avoiding both a cold standby (slow failover) and double-submission (naïve active-active). Clock discipline (chrony/PTP) keeps cross-instance and cross-venue timestamps comparable even without true colocation, which matters for both the risk engine's staleness checks and post-trade analysis.

15

Testing & simulation framework

The same compiled strategy core runs in three modes without code changes: live, shadow (computes and logs intended orders against live data without submitting), and replay (against historical data with a controllable virtual clock). This parity is only possible because of the trait boundaries in §9 and §17 — swap the `VenueAdapter` implementation and everything upstream is unaware.

- **Historical replay corpus** — timestamped weather bulletins, econ release calendars (actual vs. consensus), news headlines, and order-book snapshots, replayed at original cadence or accelerated.
- **Shadow trading** — mandatory bake-in period against live markets before any new alpha source or strategy variant is allowed to submit real orders.
- **Chaos scenarios** — feed dropout, duplicate/out-of-order messages, venue error responses, partial fills, and injected clock skew between the two hot-hot instances

- **Backtest/live parity checks** — periodically replay a recent live window through the simulator and diff intended orders against what was actually submitted; any divergence is a bug, not a modeling artifact.

16

Compliance guardrails

Kept deliberately brief here because it needs a real review, not a checklist: venue terms of service and rate limits are hard constraints, not targets to skate against; Polymarket's terms have historically restricted access for US persons and that status should be re-verified before deployment; Kalshi is a CFTC-regulated designated contract market, and quoting/order behavior must avoid patterns (layering, non-bona-fide quotes) that could be construed as manipulative under exchange rules.

This document is an engineering design, not legal, compliance, or investment advice — a review with counsel familiar with derivatives and prediction-market regulation should precede any live deployment, and venue capabilities in this document should be re-verified against current terms before implementation, since APIs and regulatory posture both evolve.